

Паттерны декомпозиции микросервисов

Microservice Architecture

REC **Проверить, идет ли запись**

Меня хорошо видно & слышно?





Сергей Прощаев

Tech Lead

Руководитель направления Java | Kotlin-разработки

- Окончил Уфимский государственный авиационный технический университет
- Более десяти лет в управлении разработкой и внедрении цифровых решений в FinTech & E-commerce

sproshchaev@gmail.com

prosoft.pages.dev

Правила вебинара



Активно
участвуем



Off-topic обсуждаем
в учебной группе
#OTUS Microservice-2026-06



Задаем вопрос
в чат или голосом



Вопросы вижу в чате,
могу ответить не сразу

Условные обозначения



Индивидуально



Время, необходимое
на активность



Пишем в чат



Говорим голосом

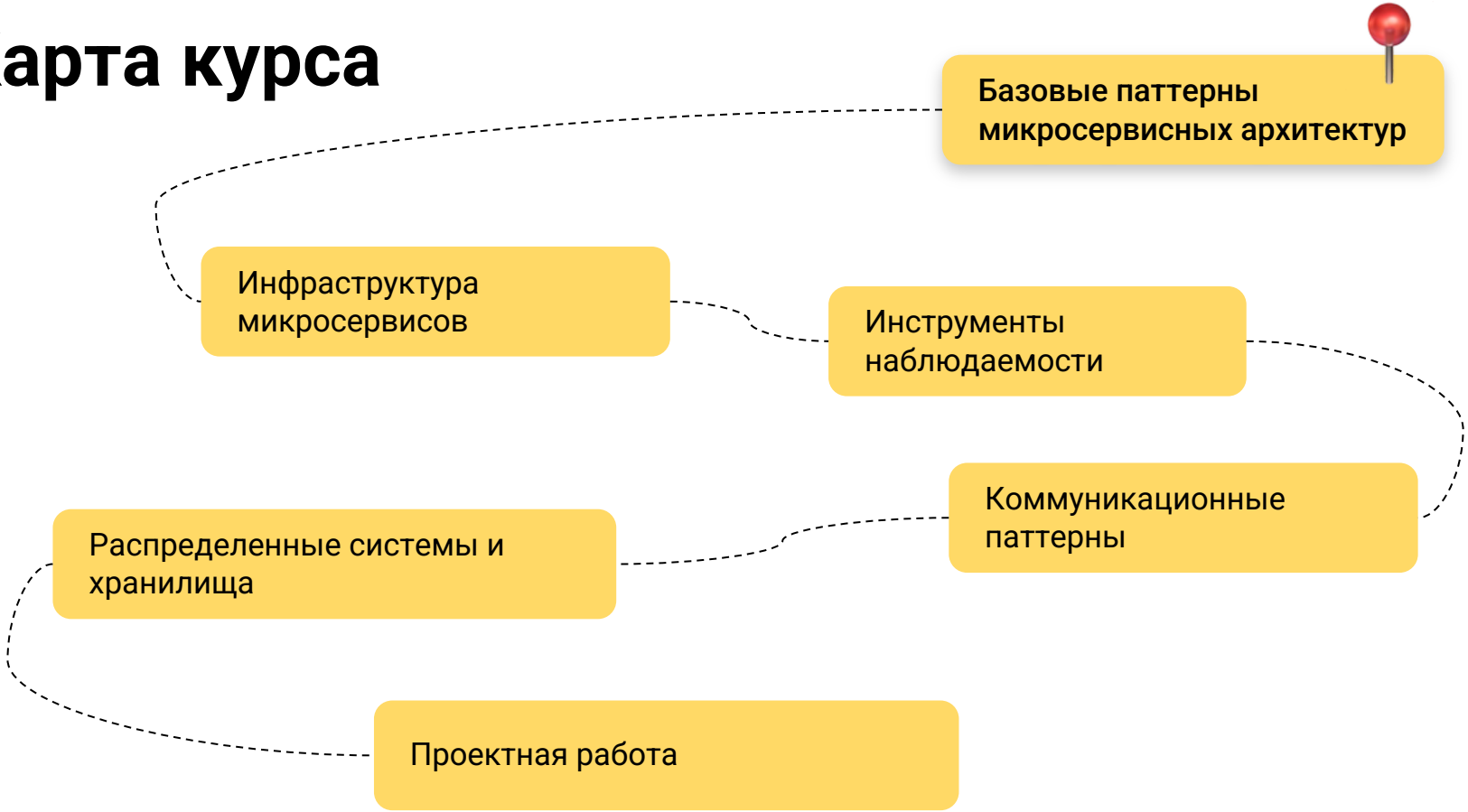


Документ



Ответьте себе или
задайте вопрос

Карта курса


Базовые паттерны микросервисных архитектур
Инфраструктура микросервисов
Инструменты наблюдаемости
Коммуникационные паттерны
Распределенные системы и хранилища
Проектная работа

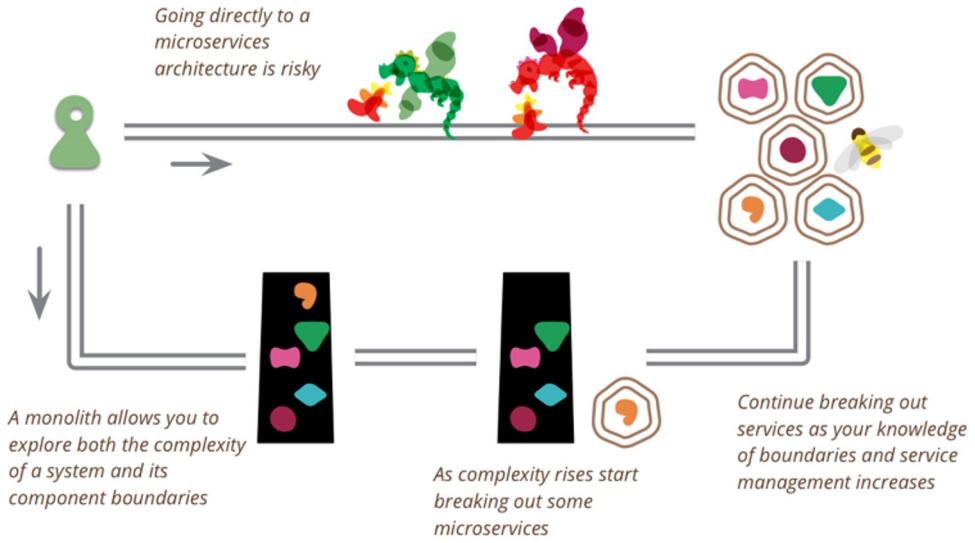
Маршрут вебинара

- Пользовательский сценарий
- Модель предметной области и системные операции
- Разбиение по агрегатам
- Функциональная модель
- Разбиение по бизнес-возможностям
- Разбиение по транзакциям

Monolith first

Monolith first

1. Относительно небольшая команда
2. Относительно небольшая (или равномерная) нагрузка
3. Высокая неопределенность



https://martinfowler.com/bliki/MonolithFirst.html

Monolith first

- Почти все успешные истории микросервисов начались с монолита, который стал слишком большим и был разрушен.
- Почти во всех известных мне случаях, когда микросервисная система строилась с нуля, это привело к серьезным проблемам.
- Не следует начинать новый проект с микросервисами, даже если вы уверены, что ваше приложение будет достаточно большим, чтобы оно того стоило.

(c) Мартин Фаулер

Причины начать с монолита

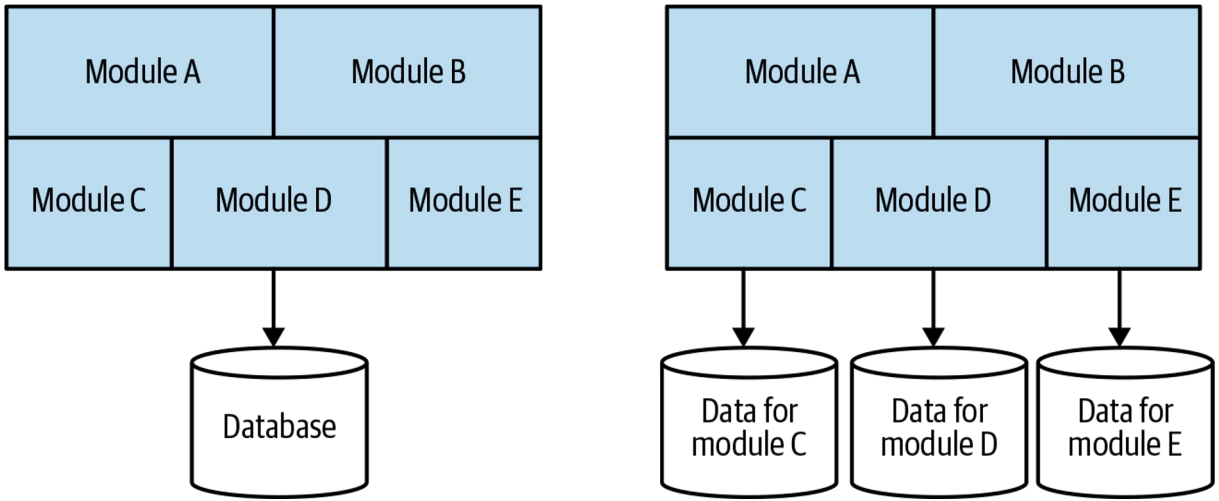
- **YAGNY** (You Aren't Gonna Need It) - Зачем сразу создавать сложную систему. Вдруг она окажется бесполезной.
- Ограниченные контексты
  - Успех микросервисной архитектуры во многом зависит от того насколько правильно вы определили границы сервисов
  - На первых этапах работы сделать это затруднительно
  - Проводить рефакторинг удобнее в монолите



Каким быть должен монолит для дальнейшего разделения на микросервисы?

Модульный монолит

- **Модульный монолит** – монолит, который деплоится как единое целое, содержит в себе всю функциональность, но при этом внутри имеется строгое разделение на независимые, слабо связанные модули.



Плюсы

- Простота разработки, легко отлаживать
- Простота развертывания и обновления. Накладные расходы на CI/CD малы
- Когда предметная область не ясна, легко изменять модель предметной области и декомпозицию
- Плавный переход к микросервисной архитектуре
- Разделение ответственности
- Четкие границы модуля
- Интерфейс (публичное API)
- Модули слабо связаны друг с другом

Что дальше делать с монолитом?

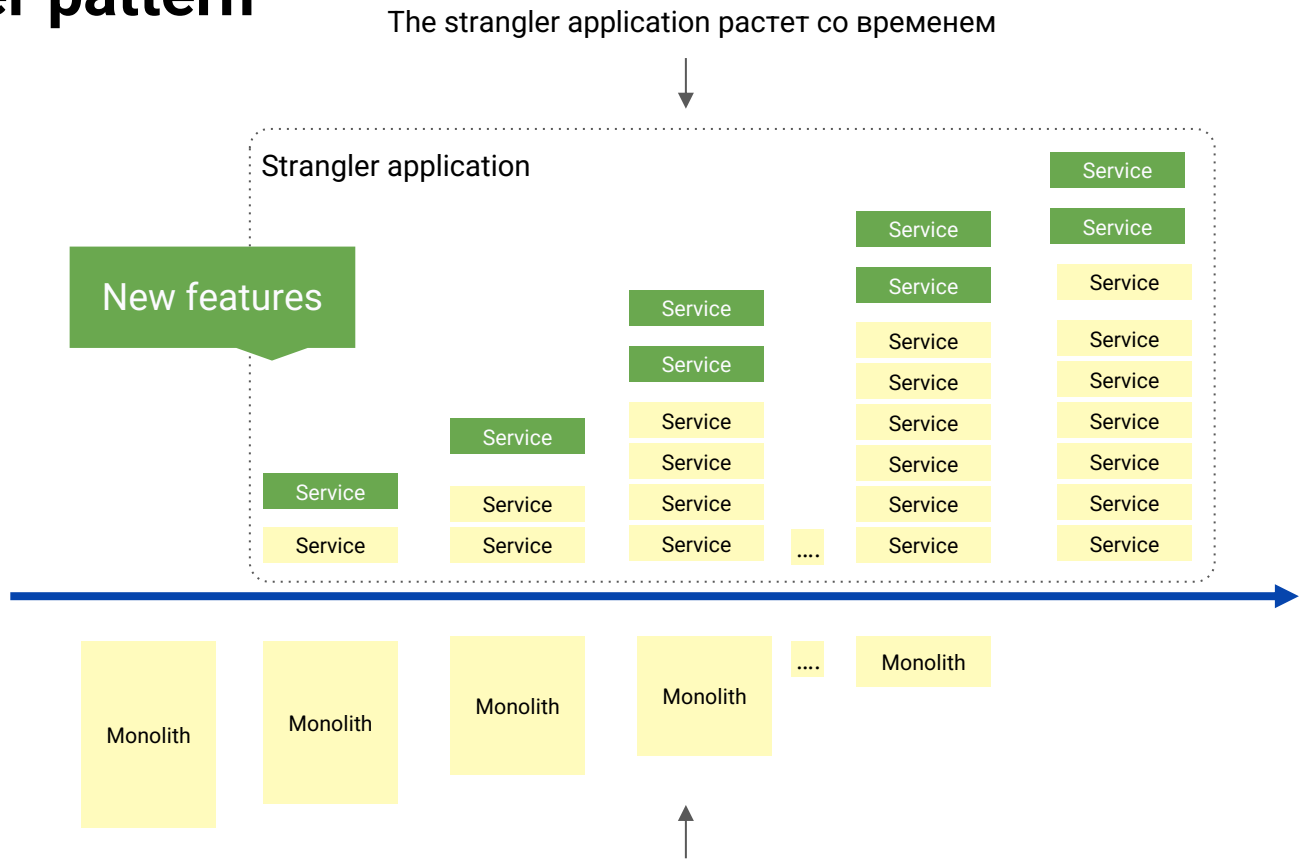
- Тщательно спроектировать монолит и в дальнейшем разбить его на куски.
- Отщипывать с краев - выделять микросервисы из монолита по мере их выявления.
- Жертвенная архитектура (Sacrificial Architecture), прототип.
    - Полностью заменить имеющийся монолит на микросервисы.
    - Рассматривать монолит как реализацию, которую мы должны будем рано или поздно выбросить, но при этом ничего лучше в текущий момент создать не можем.



Какой основной паттерн будем применять для разделения монолита на микросервисы?

Ответ

Strangler pattern



Декомпозиция на микросервисы

Как декомпозировать на сервисы?

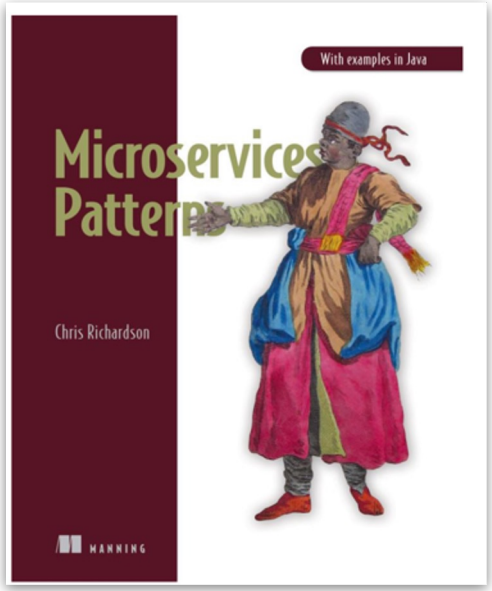
1. Описать сценарии использования
2. Определить системные операции(действия) с приложением
3. Построить доменную модель (модели)
4. Построить сервисную модель
5. Протестировать сервисную модель
6. Повторить п.3 – п.5 пока не станет хорошо
7. Ничего ли мы не забыли?

Домен – область деятельности. Пример: Доставка, Кухня.

Пример

Система вроде “Яндекс-еда” – рестораны, курьеры, заказ блюд

FTGO



Как декомпозировать на сервисы?



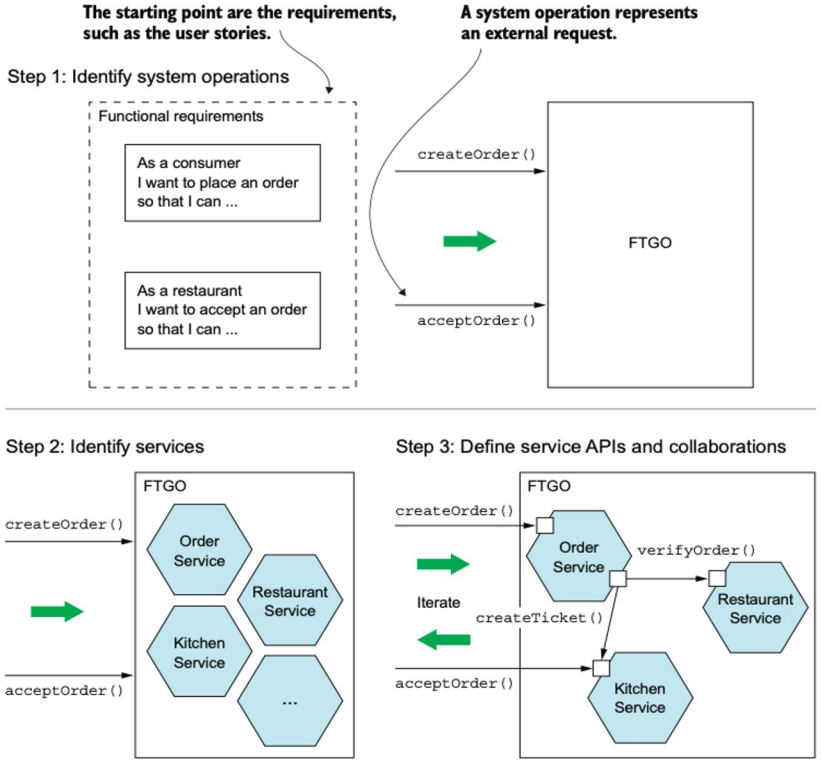


Figure 2.5 A three-step process for defining an application's microservice architecture

Пользовательский сценарий

Пользовательский сценарий

Чаще всего пользовательские сценарии описывают в формате BDD*/User Story.



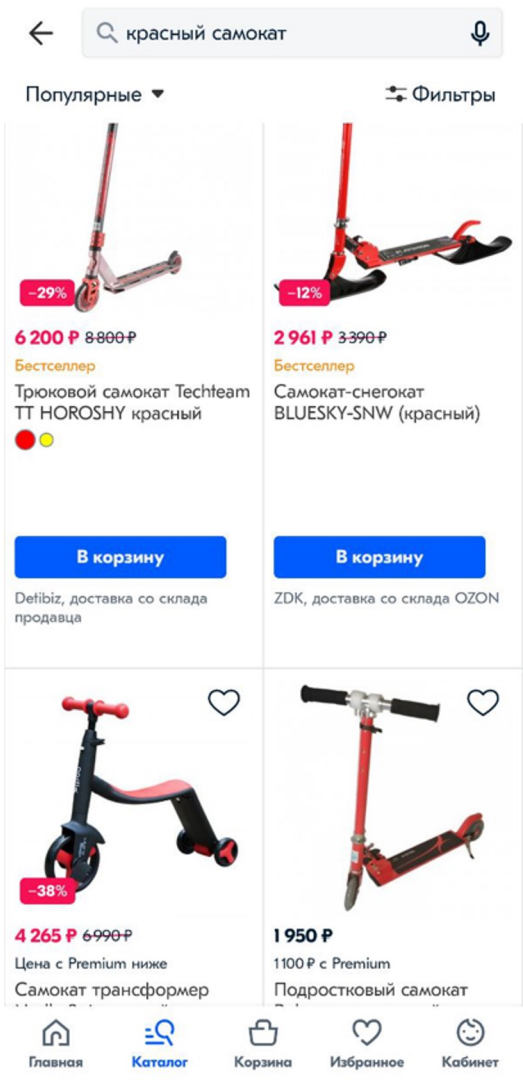
Википедия

* Behavior-Driven Development

Пользовательский сценарий

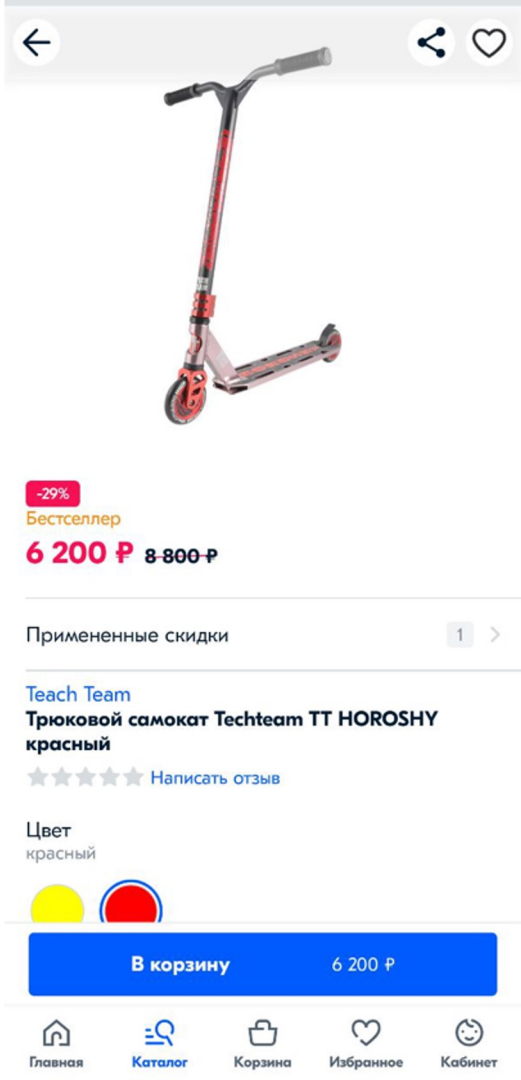
Есть авторизованный клиент
И есть интернет-магазин "Гозон"
И клиент вводит в строке поиска "Красный самокат"
Тогда появляется список товаров, подходящих под запрос клиента
И в карточке товара пользователь видит

1. название
2. цену
3. изображение
4. скидку
5. доступность в разных магазинах



Пользовательский сценарий

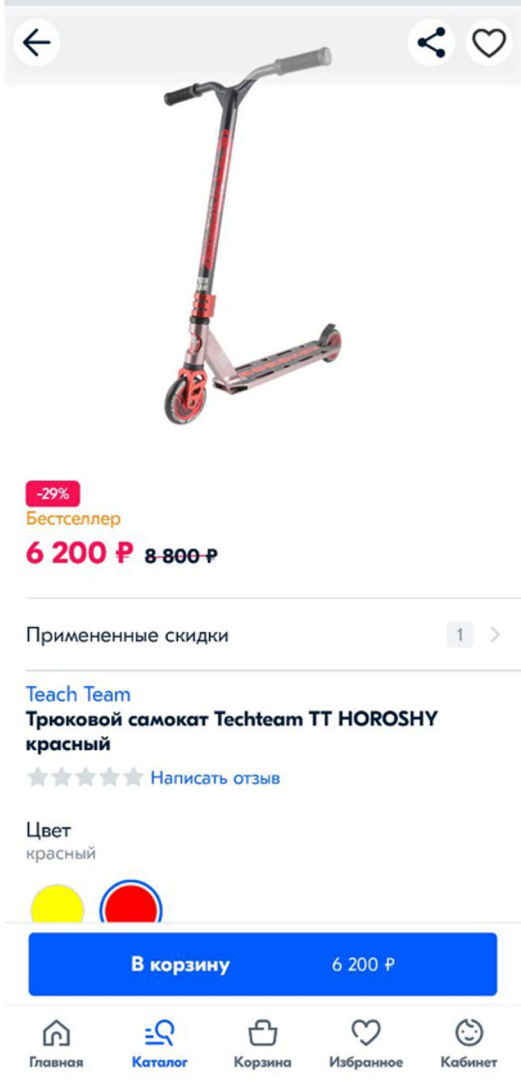
Когда клиент нажимает карточку на товар
Тогда клиент переходит на страничку с описанием товара



Пользовательский сценарий (User Story)

Я как клиент могу зарегистрироваться на сайте.
Я как клиент могу просматривать страницы товара с описанием.
Я как клиент могу положить товар в корзину, оформить заказ, оплатить и получить свой заказ.

Описывается завершенный функционал для пользователя

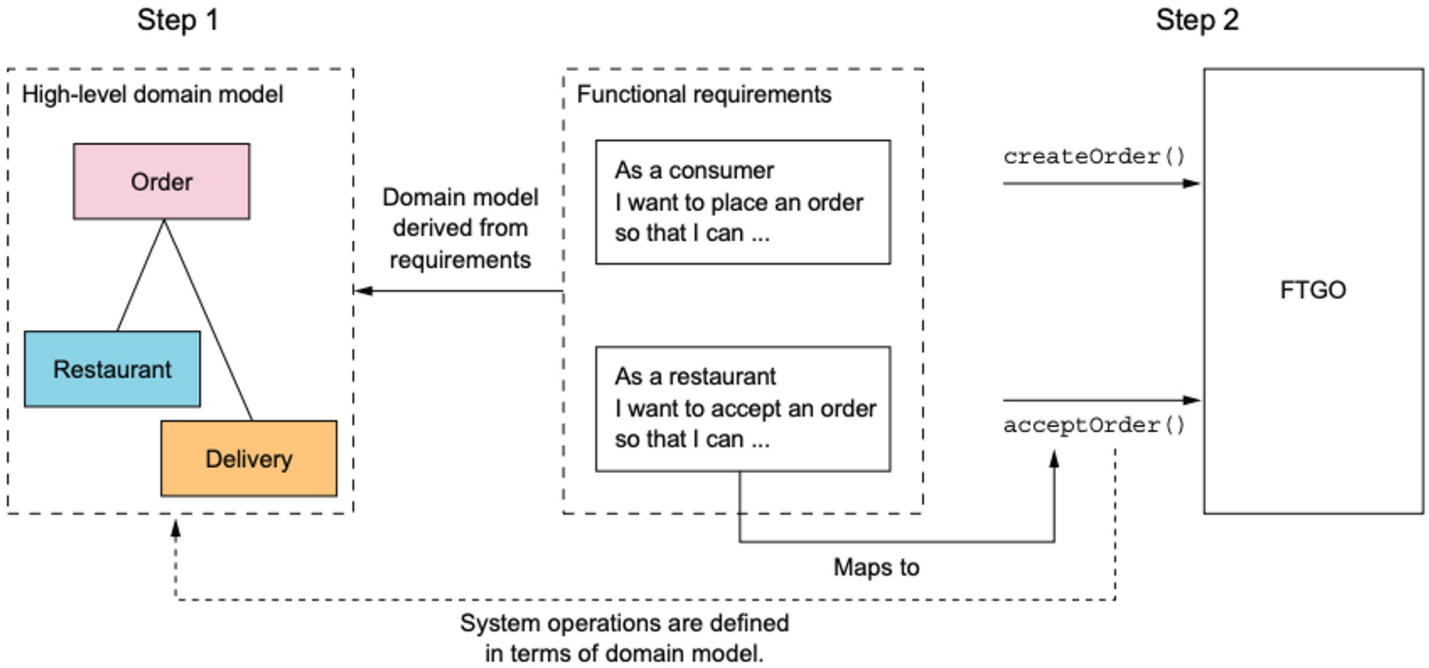


Модель предметной области и системные операции

Системные операции

- **Системные операции** – это действия, которые совершает пользователь и приложение в рамках пользовательского сценария.
- **Модель предметной области** – концептуальная модель. Это совокупность понятий предметной области и отношений между ними, а также набор инвариантов и ограничений предметной области, важных при решении конкретного спектра задач.

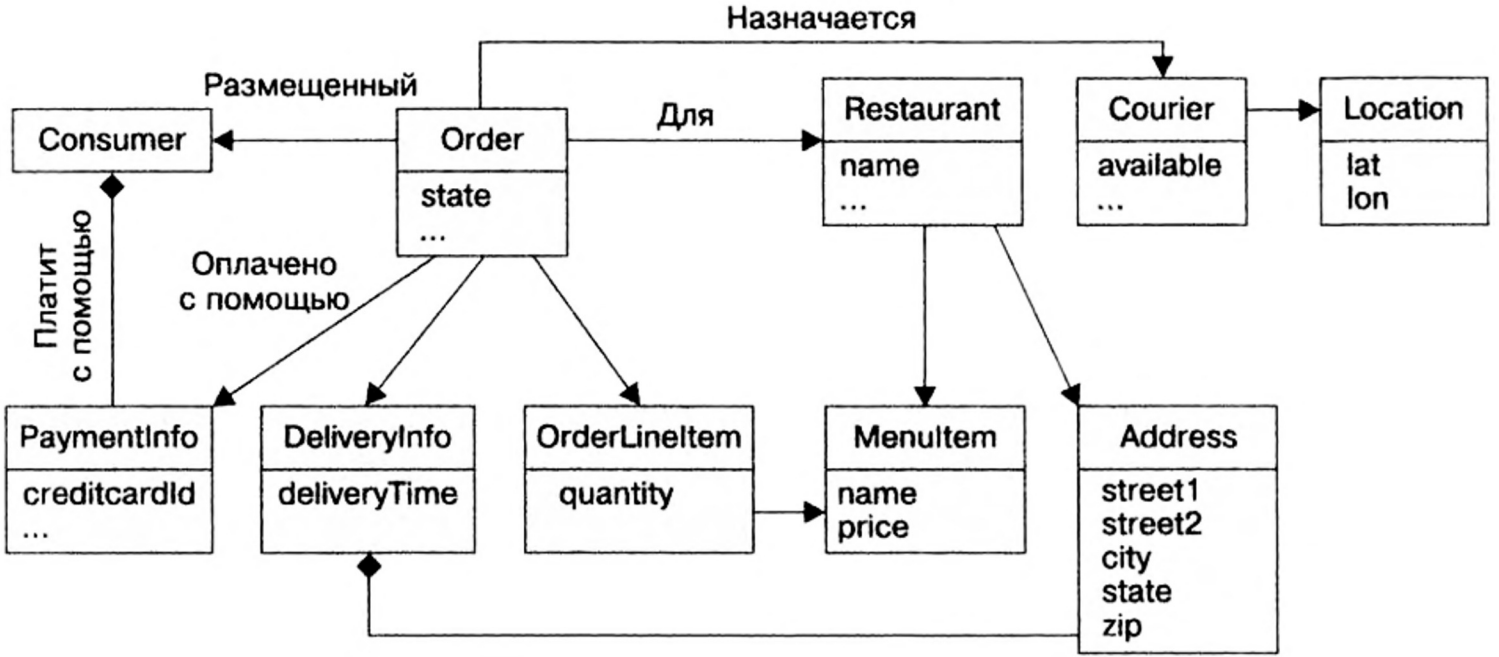
Системные операции



Доменная модель - существительные

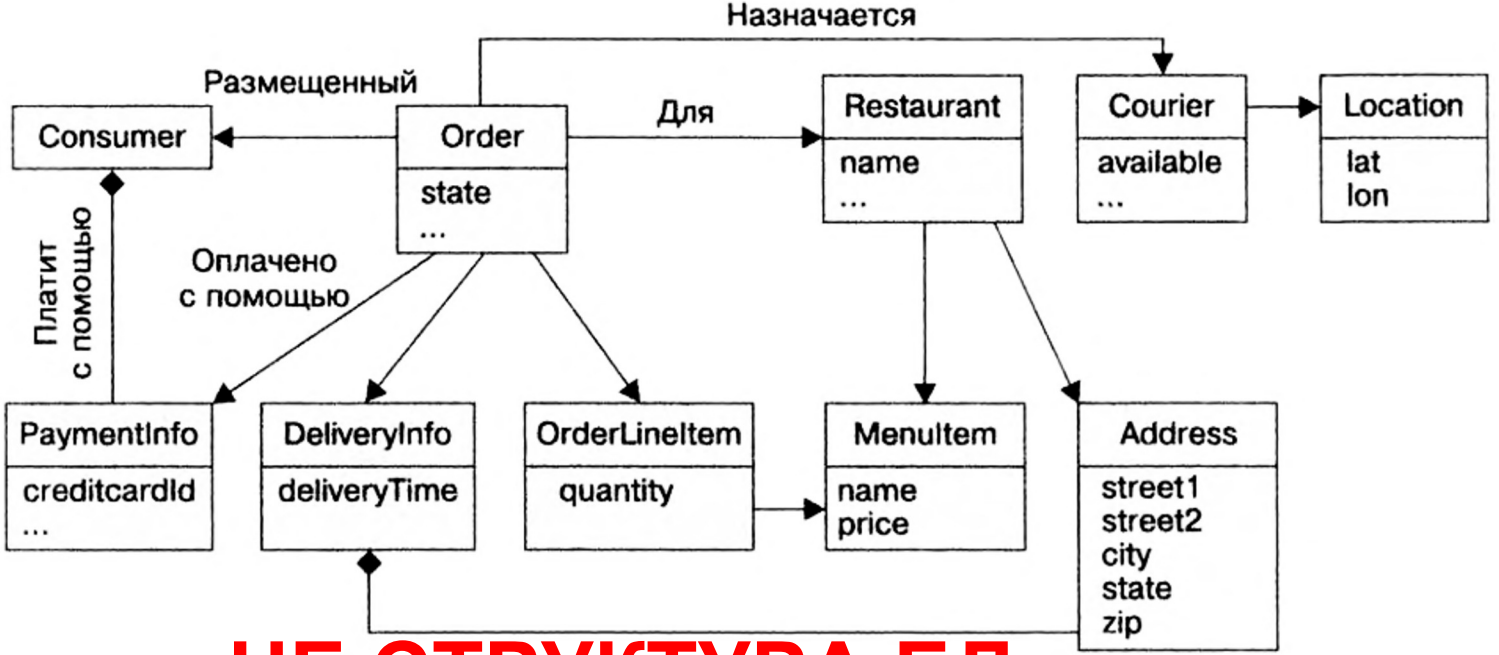
- Клиент размещает заказ
- Клиент получает данные о продукте
- Клиент добавляет продукт в заказ
- Клиент может убрать продукт из заказа
- Клиент может оформить заказ
- Клиент выбирает место доставки в заказе
- Клиент выбирает способ оплаты в заказе
- Клиент выбирает время доставки в заказе
- Клиент проводит оплату заказа
- Ресторан принимает заказ
- Приложение резервирует курьера для доставки заказа

Доменная модель - схема



Доменная модель - схема

НЕ МОДЕЛЬ ДАННЫХ



НЕ СТРУКТУРА БД

Системные операции

- *команды* — системные операции для создания, обновления и удаления данных;
- *запросы* — системные операции для чтения (запрашивания) данных

Системные операции - глаголы

- Клиент размещает заказ
- Клиент получает данные о продукте
- Клиент добавляет продукт в заказ
- Клиент может убрать продукт из заказа
- Клиент может оформить заказ
- Клиент выбирает место доставки в заказе
- Клиент выбирает способ оплаты в заказе
- Клиент выбирает время доставки в заказе
- Клиент проводит оплату заказа
- Ресторан принимает заказ
- Приложение резервирует курьера для доставки заказа

Системные операции

Действующее лицо	История	Команда	Описание
Клиент	Create Order	createOrder()	Создает заказ
Ресторан	Accept Order	acceptOrder()	Указывает на то, что ресторан принял заказ и обязуется приготовить его к заданному времени
	Order Ready for Pickup	noteOrderReadyForPickup()	Указывает на то, что заказ готов к доставке
Курьер	Update Location	noteUpdatedLocation()	Обновляет текущее местоположение курьера
	Delivery picked up	noteDeliveryPickedUp()	Указывает на то, что курьер взял заказ
	Delivery delivered	noteDeliveryDelivered()	Указывает на то, что курьер доставил заказ

Системные операции - детализация

Операция	createOrder (ID клиента, способ оплаты, адрес доставки, время доставки, ID ресторана, позиции заказа)
Возвращает	orderId...
Предварительные условия	Клиент существует и может размещать заказы. Позиции заказа соответствуют пунктам меню ресторана. Адрес и время доставки выполнимы для ресторана
Окончательные условия	Банковская карта клиента позволила снять сумму заказа. Заказ был создан в состоянии PENDING_ACCEPTANCE

Операция	acceptOrder (restaurantId, orderId, readyByTime)
Возвращает	—
Предварительные условия	order.status равно PENDING_ACCEPTANCE. Курьер доступен для доставки заказа
Окончательные условия	Состояние order.status поменялось на ACCEPTED. Время order.readyByTime поменялось на readyByTime. Курьер назначен для доставки заказа

Слайд с тезисами

1. Рассмотрели описание пользовательских сценариев.
2. Рассмотрели понятие **предметной области** (домен).
3. Рассмотрели **системные операции.**

Как декомпозировать на сервисы?

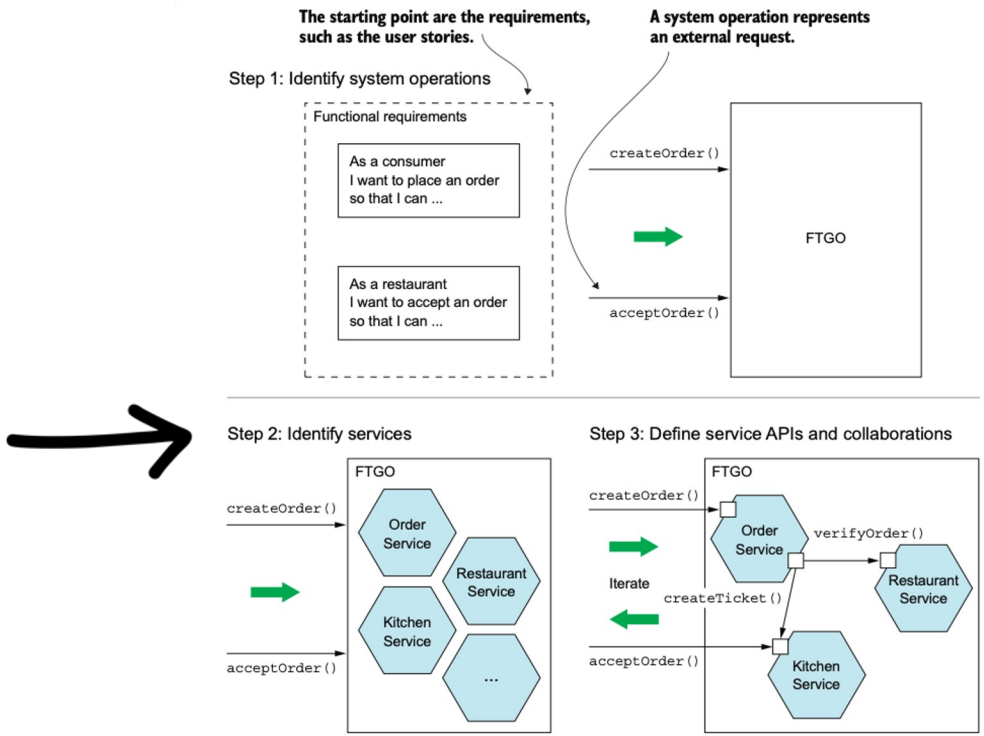
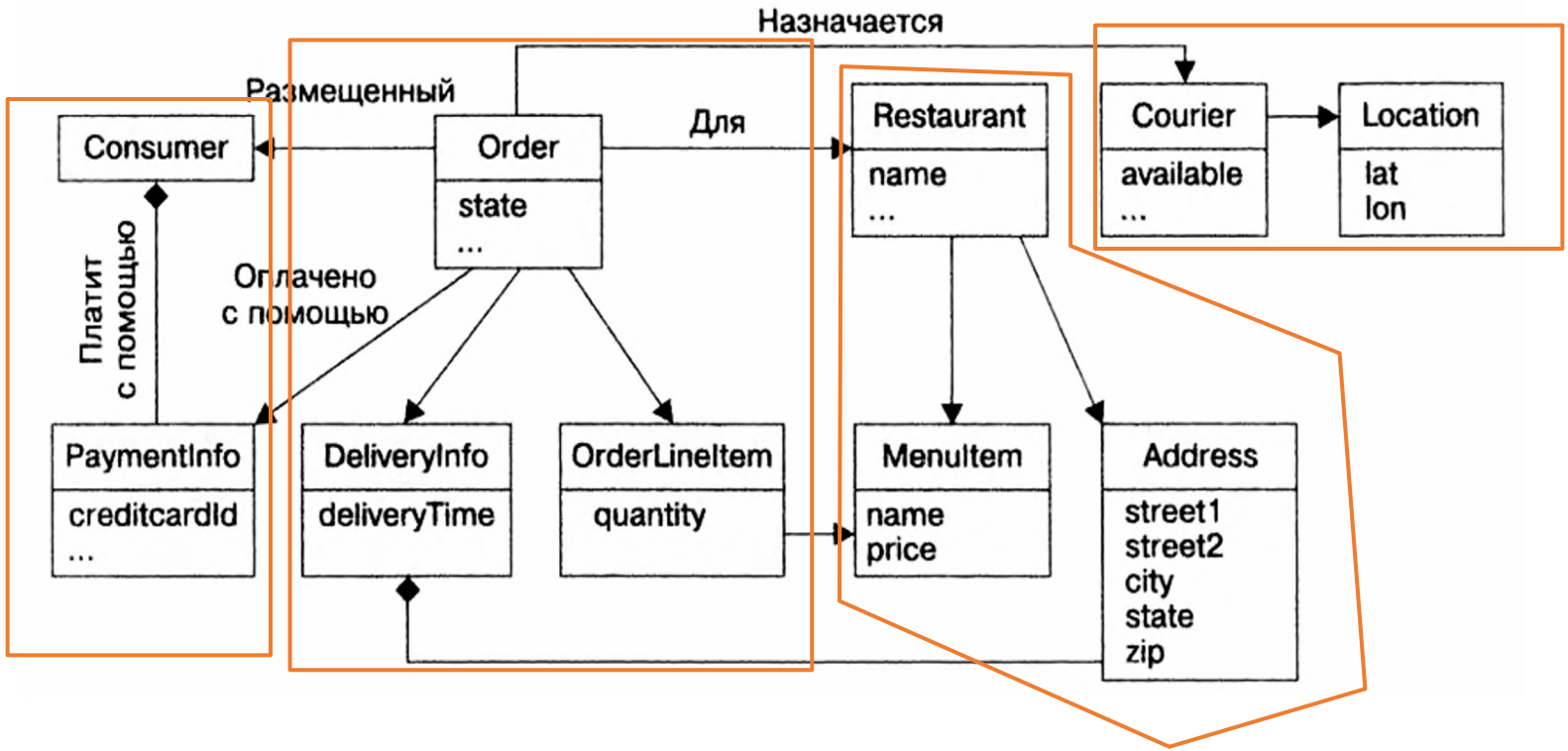


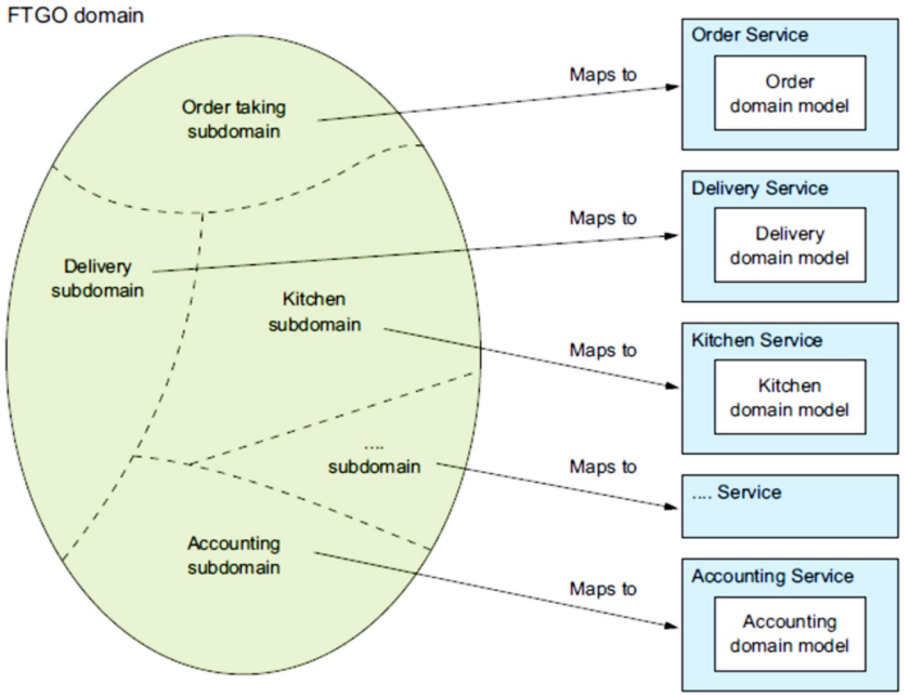
Figure 2.5 A three-step process for defining an application's microservice architecture

Разбиение по агрегатам

Доменная модель - схема



Нулевая итерация модели предметной области

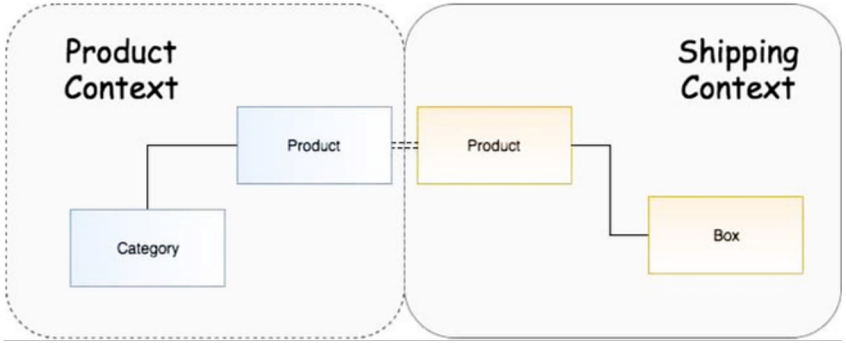


Эрик Эванс - DDD

- В целом мы можем добавлять сущностей или делать модель более детализированной.
- Чтобы перейти к моделированию сервисов, для сложных предметных областей, мы должны каким-то образом провести границы, по которым сервисы будут проходить.
- В DDD есть соответствующие паттерны для проведения таких границ – модули, агрегаты и сервисы.
- Нарисовать единую картину предметной области - очень сложно и часто невыполнимо.
- Давайте разобьем на подобласти и в каждой из них будет свой язык (Ubiquitous Language) и свои части данных.

Ограниченный контекст (Bounded context)

- Выделенные поддомены ограничивают при реализации системы (ограниченный контекст).
- Ограниченные контексты используются для создания переборок в больших и сложных системах.



- Подобно переборкам в корпусе корабля, которые предотвращают затопление всего корабля, переборка в системе предотвращает утечку ненужной сложности за пределы контекстной границы.

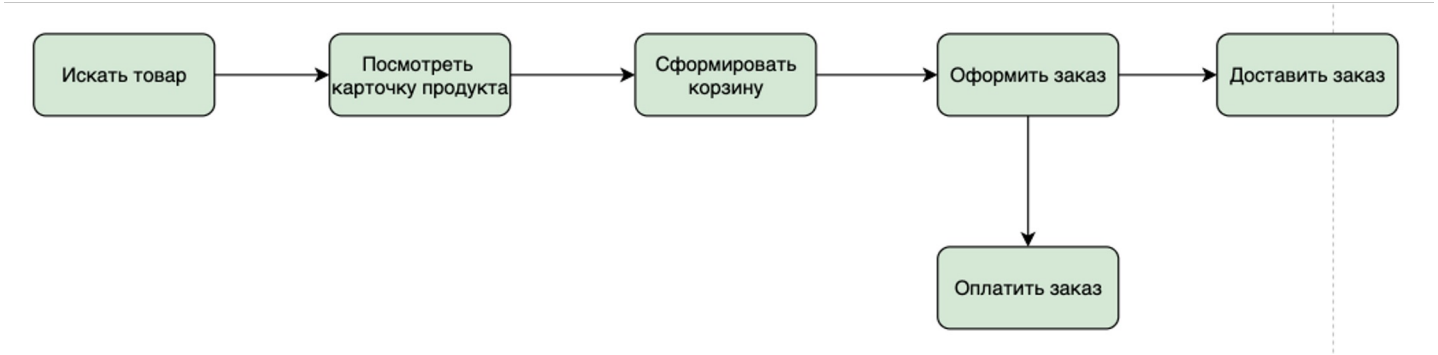
Функциональная модель

Функциональное моделирование

- Моделировать можно не только как набор сущностей, но и как просто некий флоу – в «функциональном»/процедурном стиле.

Функциональное моделирование

- В качестве действий использовать использовать какие-то «крупные» бизнес-действия.



Функциональное моделирование

- В качестве первого приближения для разбиения на сервисы использовать каждый блок – как отдельный сервис или набор связанных сервисов.
- В данном случае сервисами в сервисной модели будут:
  - Поиск блюда
  - Каталог блюд
  - Сформировать корзину
  - Оформление заказа
  - Доставка заказа
  - Оплата заказа
  - Отмена заказа

Разбиение
по бизнес-возможностям

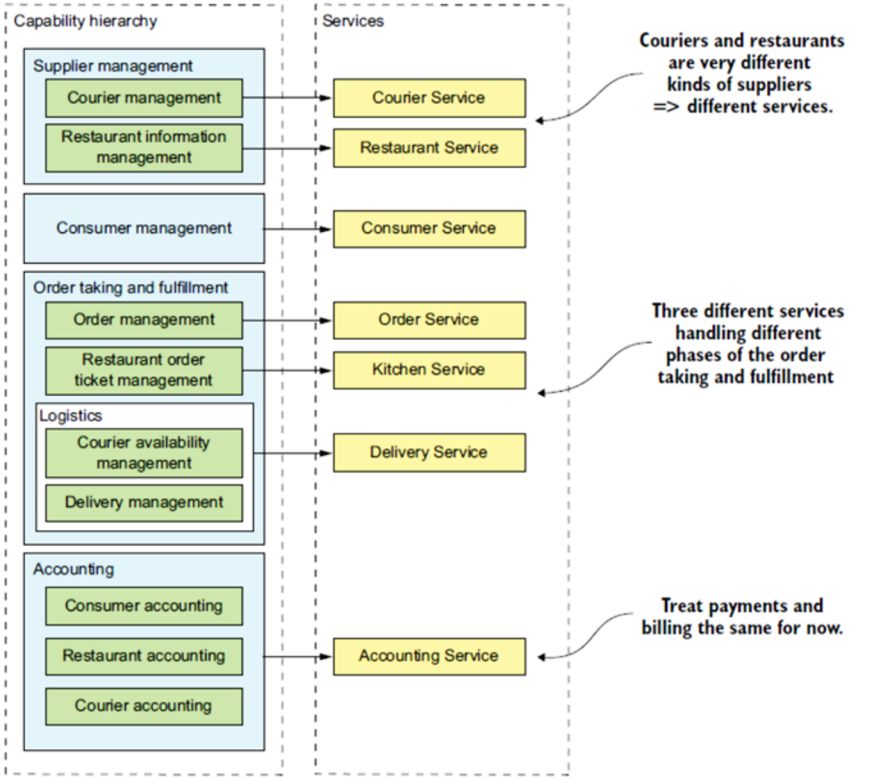
Бизнес-возможности

- Бизнес-возможности организации описывают то, чем она является.
- Обычно они стабильны, в отличие от того, как организация ведет свой бизнес (этот аспект со временем меняется, иногда до неузнаваемости).

Бизнес-возможности

- Управление поставщиками:
  - управление курьерами
  - управление информацией о ресторанах
- Управление клиентами
- Прием и выполнение заказов:
  - управление заказами со стороны клиентов
  - управление заказами в ресторане
  - логистика
  - управление доступностью курьеров
- Бухгалтерский учет

Бизнес-возможности



Разбиение
по транзакциям

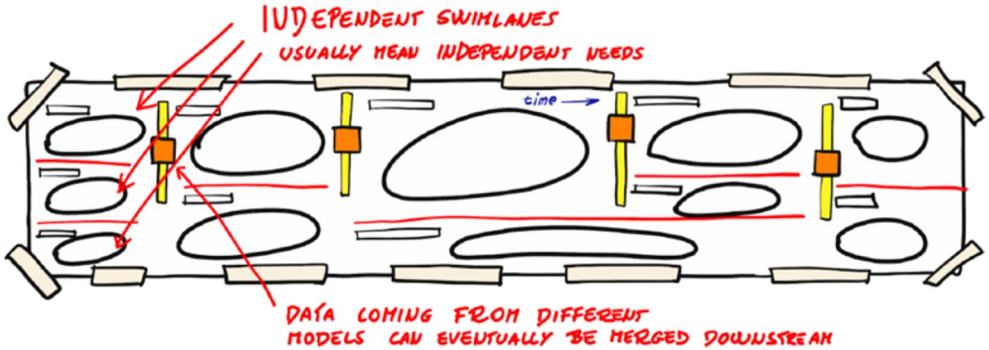
Транзакции и MSA

- Важное ограничение MSA: с базой данных связан только один сервис.
- Если транзакция затрагивает несколько микросервисов, то возникает техническая проблема организации транзакции.
- Разбиваем систему на микросервисы таким образом, чтобы любая транзакция осуществлялась в пределах одного компонента (сцепленность по транзакциям).

Event Storming

Event storming

- Для относительно небольших проектов хватает функционального и ООП моделирования, чтобы перейти к модели сервисов.
- Для крупных проектов – обычно не хватает. В этом случае может подойти «гибридный» и более современный подход – event storming.



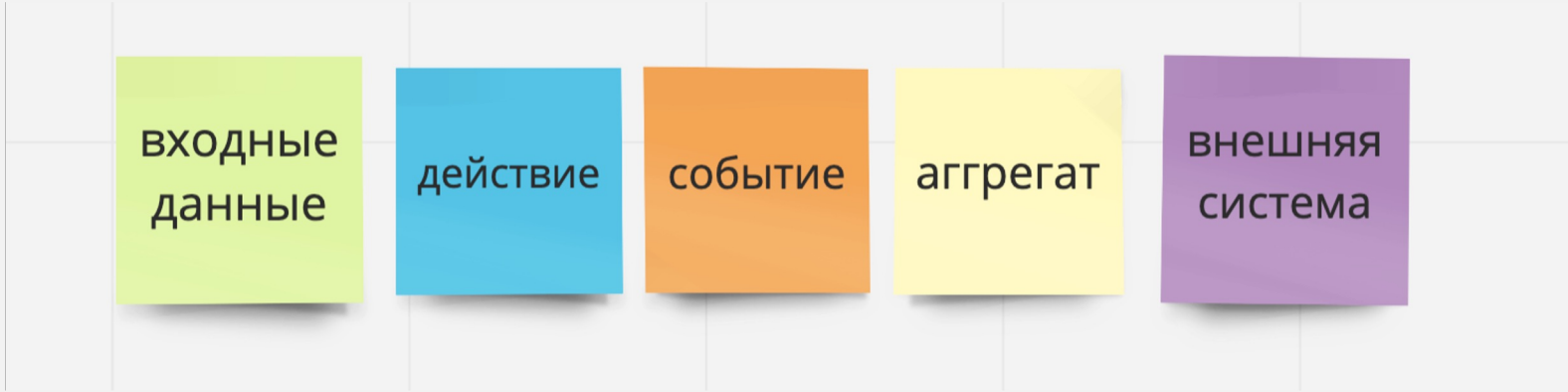
https://www.eventstorming.com/book/

Event storming

- Event storming – это собрание, на котором разработчики и бизнес вместе выясняют, как же выглядит предметная область.
- Обычно выделяется большое помещение, **очень** большая доска (или стена), **очень** много стикеров и маркеров.
- И в течение дня или двух дней происходит моделирование предметной области.

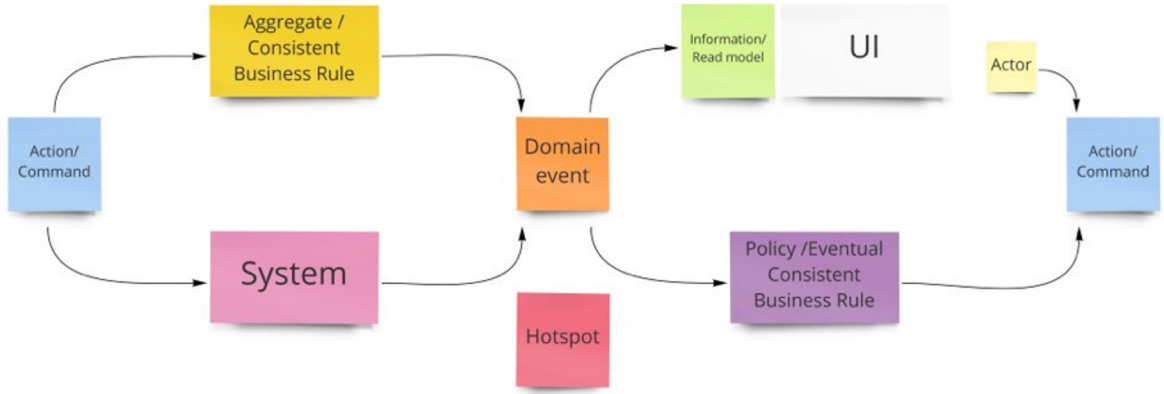
Event storming

- Отражаем на доске бизнес процесс в виде набора стикеров

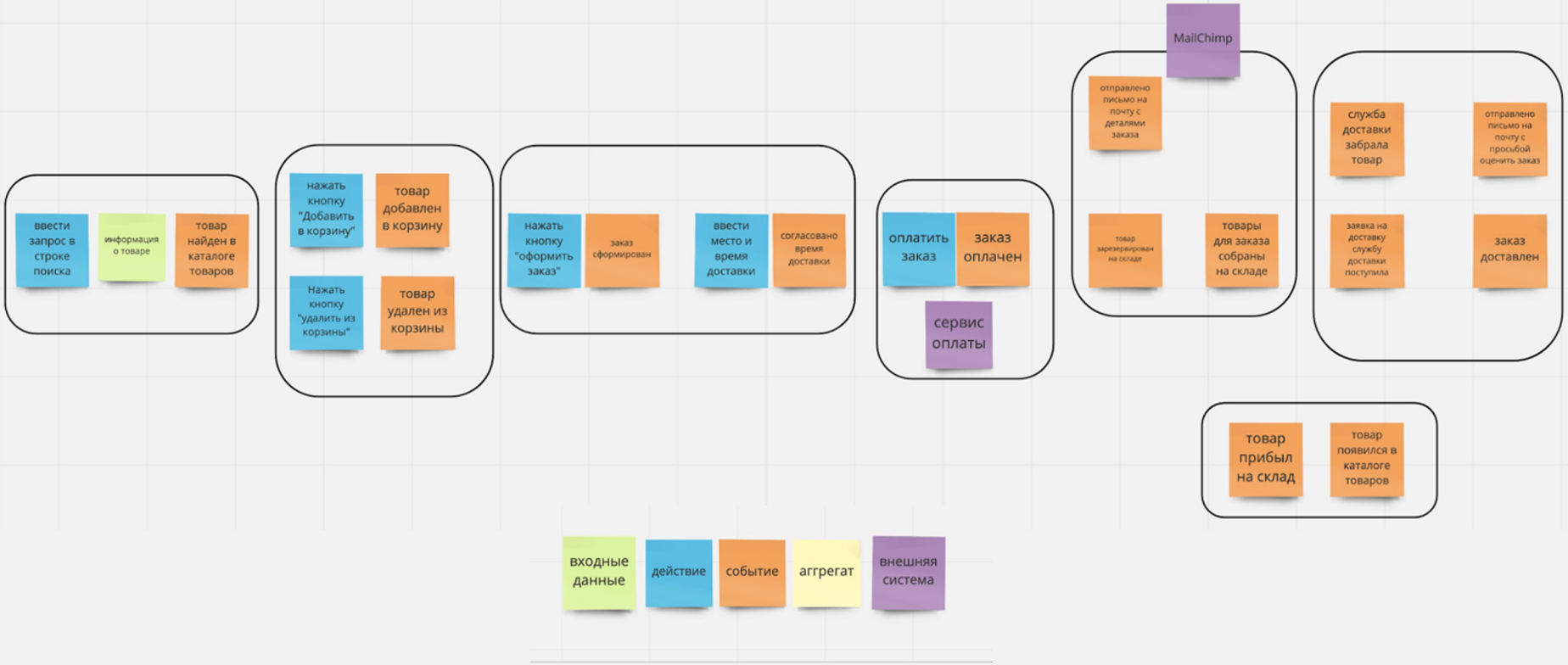


Event storming

- Представляем бизнес процесс в виде набора действий, запросов, событий, акторов, агрегатов и внешних систем.
- Пытаемся провести границы вокруг сущностей, которые потом станут границами команд и микросервисов (или группы микросервисов).
- В ограниченный контекст включается: описание, бизнес правила, запроса + команды + события, зависимости от других контекстов



Event storming



https://miro.com/app/board/o9J_kvo2lAo=/

Event storming. Проблемы и особенности

- Очень многословен. Фактически для создания подробного event storming-а требуется написать спецификацию всего кода в терминах DDD. Даже для небольших проектов он может быть многословен. (https://github.com/ddd-by-examples/library)
- Может содержать ошибки. Кто-то какие-то события не вспомнил, кто-то что-то неправильно написал и т.д. В целом это больше инструмент коммуникации, нежели подробного и точного моделирования.
- Результаты моделирования остаются на бумаге (точнее на стене) и ими дальше не руководствуются в принятии архитектурных решений.

Event storming. Проблемы и особенности

- В рамках Event Storming лучше моделировать большие bounded context-ы (в среднем 1 на 1 команду).
- Моделирование маленьких ограниченных контекстов и деление на сервисы/микросервисы в рамках сессии event storming кажется не продуктивным.

Как декомпозировать на сервисы?

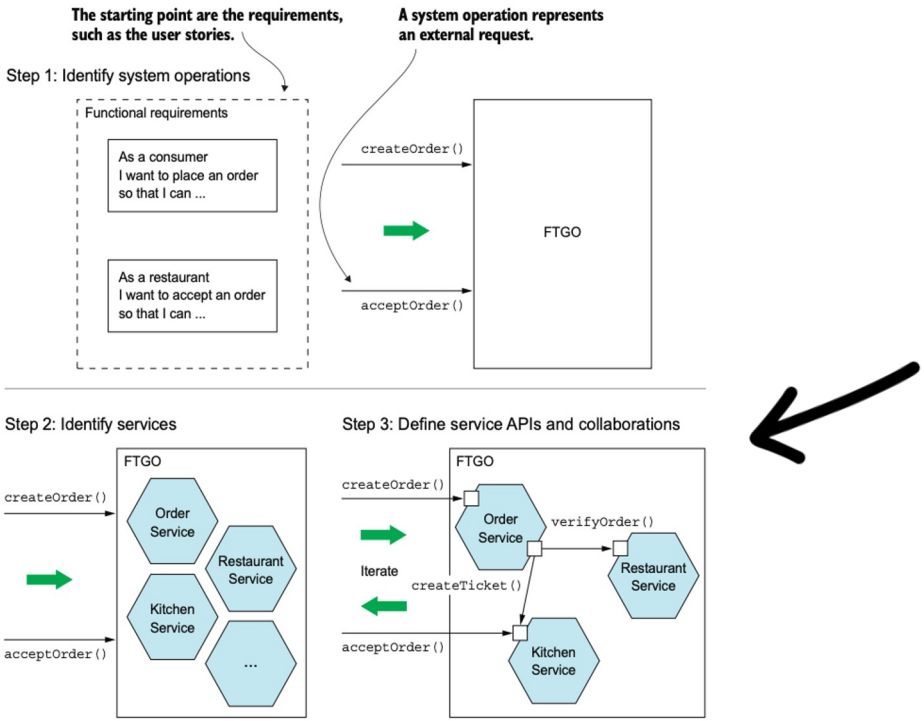


Figure 2.5 A three-step process for defining an application's microservice architecture

API и описание сервисов

Моделирование

- Какой бы способ мы ни выбрали, в любом случае надо понимать, что из модели мы получаем лишь первое приближение для разбиения сервисов.
- И в дальнейшем разделение на сервисы мы можем делать исходя из нефункциональных требований – требований производительности, безопасности, масштабируемости и т.д и т.п.

Как моделируем сервисы?

- Прописываем системные действия для каждого сервиса уже в виде API
- Пробегаемся по основным сценариям и смотрим, не забыли ли мы что-нибудь и насколько хорошее и полное API у нас получается
- Проходим по всем сервисам и определяем ответственность сервиса, зависимости между сервисами, пытаемся прописать конкретное API и выбираем протокол для взаимодействия
- Находим проблемы в текущей модели разбиения
- Корректируем модель и разбиение
- Повторяем

Системные действия в модели сервисов

Клиент
GET /api/v1/client/ { delivery_address, payment_method }

Продукт
GET /api/v1/products/search?q=Гамбургер&limit=20
GET /api/v1/products/{id}/

Корзина
POST /api/v1/cart/products/ {id, quantity: 1}
DELETE /api/v1/cart/products/{id}/
PUT /api/v1/cart/products/{id}/ {quantity: 2}

Заказ
POST /api/v1/order/ { delivery_time, delivery_address, payment_method }
POST /api/v1/order/pay/ - редирект на сервис оплаты

Ресторан
POST /api/v1/restourant/prepare/ { order_id }

Какие методы еще нужны исходя из сценария?

Продукт
GET /api/v1/products/catalog/?category_id=23&subcategory_id=43&q=Бургер
POST /api/v1/products/catalog/filter

Корзина
GET /api/v1/cart/products/ - забыли получить все продукты в корзине

Заказ
GET /api/v1/orders/ - список заказов

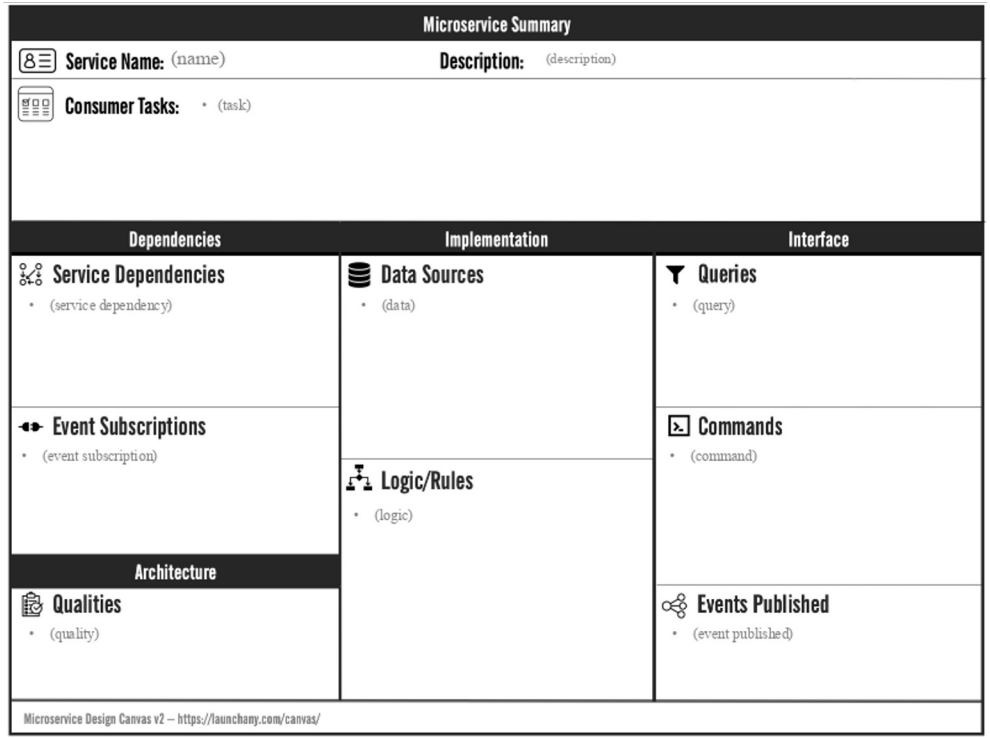
Словари
GET /api/v1/delivery_types/ - список типа доставок
GET /api/v1/payment_methods/ - список типа платежей

Описание сервисов

- Для более формального описания сервисов может подойти один из шаблонов для описания:
  - MicroservicesCanvas
  - BoundedContextCanvas

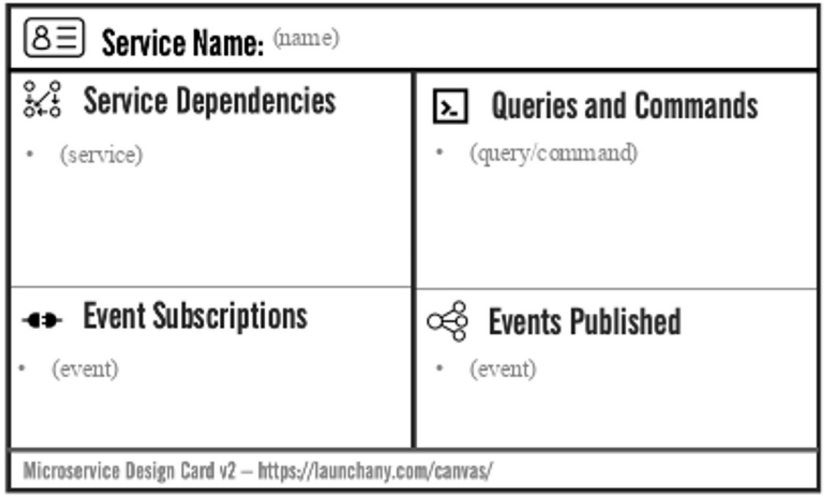
Microservices Canvas

Формат описания сервисов от launchany (https://launchany.com/microservice-design-canvas/)



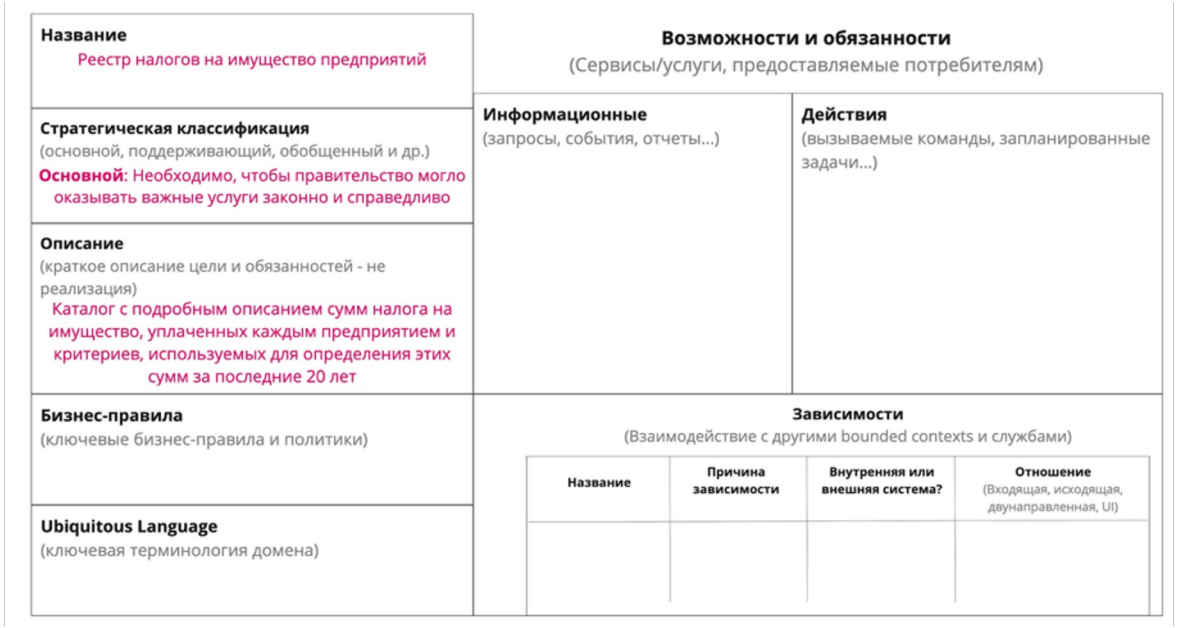
Microservices Canvas

Более простая форма описания сервиса



Bounded Context Canvas

Bounded Context Canvas – похож на канвас про микросервисы, но более DDD ориентированный и построение bounded context-ов, его можно использовать для описания сервисов



Название

Реестр налогов на имущество предприятий

Возможности и обязанности

(Сервисы/услуги, предоставляемые потребителям)

Стратегическая классификация

(основной, поддерживающий, обобщенный и др.)

Основной: Необходимо, чтобы правительство могло оказывать важные услуги законно и справедливо

Информационные

(запросы, события, отчеты...)

Действия

(вызываемые команды, запланированные задачи...)

Описание

(краткое описание цели и обязанностей - не реализация)

Каталог с подробным описанием сумм налога на имущество, уплаченных каждым предприятием и критериев, используемых для определения этих сумм за последние 20 лет

Бизнес-правила

(ключевые бизнес-правила и политики)

Зависимости

(Взаимодействие с другими bounded contexts и службами)

Название	Причина зависимости	Внутренняя или внешняя система?	Отношение (Входящая, исходящая, двунаправленная, UI)

Ubiquitous Language

(ключевая терминология домена)

https://habr.com/ru/company/oleg-bunin/blog/500506/

Сервис “Продукт”

Название:
Продукт

Запросы:
- Поиск продукта GET /api/v1/search/?q=…
- Информация о продукте GET /api/v1/products/{id}
- Информация о списке продуктов
- …

Команды:
- Изменение товарной номенклатуры
  POST on /api/v1/products/{id}

События:
- **-**

Зависимости:
- Слушает события ProductReserveChanged от Склада.

Вопросы:
- Атрибуты для поиска продуктов и технология – Elastic Search/Solr для поиска использовать?

Сервис “Корзина”

Название:
Корзина

Запросы:

- Список продуктов в корзине
  GET /api/v1/cart/products/ { [product_id , quantity_id] }

Команды:

- Изменение состава продуктов в корзине
  POST /api/v1/cart/products/ {id, quantity: 1}
  DELETE /api/v1/cart/products/{id}/
  PUT /api/v1/cart/products/{id}/ {quantity: 2}

События:

- -

Зависимости:

Вопросы:

- В какой момент очищается корзина? Ее в явном виде очищает создание заказа? Типа когда создан заказ, тогда очищать? Можно ли ее сделать на клиенте?
- Зависимость от остатка на складе

Возможные изменения

Возможные изменения

- Сервис Продукт имеет смысл разделить на 2.
- Формирование корзины и оформление заказа кажется, что про одно и то же. И сервисы получаются сильно связные. Имеет смысл эти сервисы объединить
- Сервис Заказ много с кем связан и интегрирует в себя много логики, имеет смысл отдельно выделить сервис оплаты

Нужен ли отдельный сервис оплаты?

- Что будет, если из сервиса Заказов сразу ходить во внешний сервис?
- Сервис Заказов будет «знать» про внутренности внешнего сервиса оплаты.
- Если понадобится оплата вне сервиса заказов (например, если надо будет пополнять счет в личном кабинете), то мы не сможем переиспользовать код.



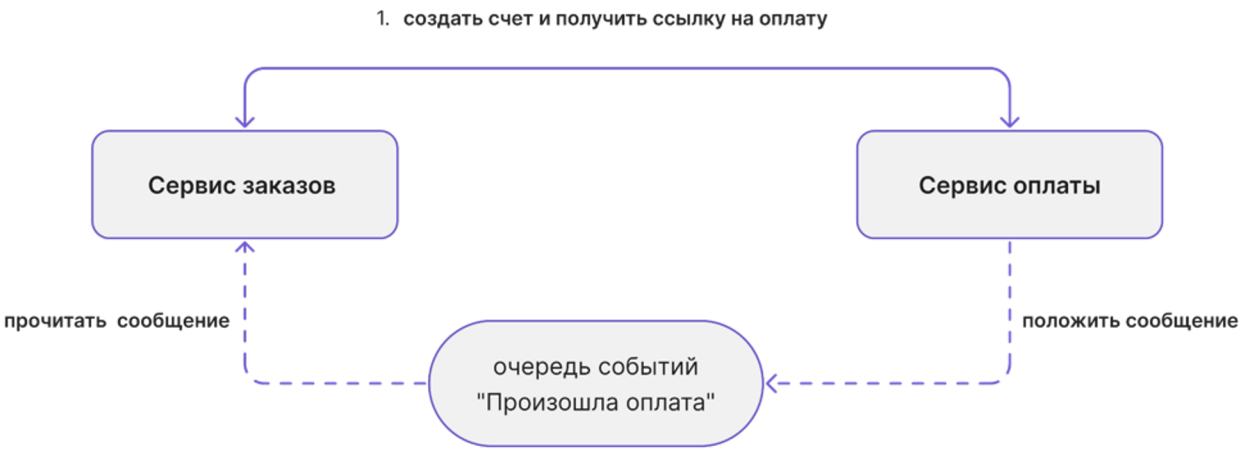
Как связать сервис оплаты и сервис заказов?

- Что будет, если мы сделаем прослойку в виде Сервиса оплаты?
- Создать счет и получить ссылку на оплату можно по API
  POST /api/v1/invoices/ { payment_type_id amount {order_id, redirect_url}}
- Без ссылки на оплату флоу работы клиента заблокирован.
- Как можно уведомить сервис Заказов, что оплата прошла?
  POST /srv/api/v1/orders/{order_id}/invoice_payed/
- В этом случае «Сервис оплаты» явно знает про «Сервис заказов» и получаем похожую проблему.



Как связать сервис оплаты и сервис заказов?

- Что будет, если мы сделаем очередь событий «Произошла оплата»?
- Сервис оплаты теперь не будет знать явно про Сервис заказов.
- Но в этом случае увеличится latency, насколько будет комфортно пользователю, что сообщение об оплате может появится не сразу?
- Кажется, что ок.



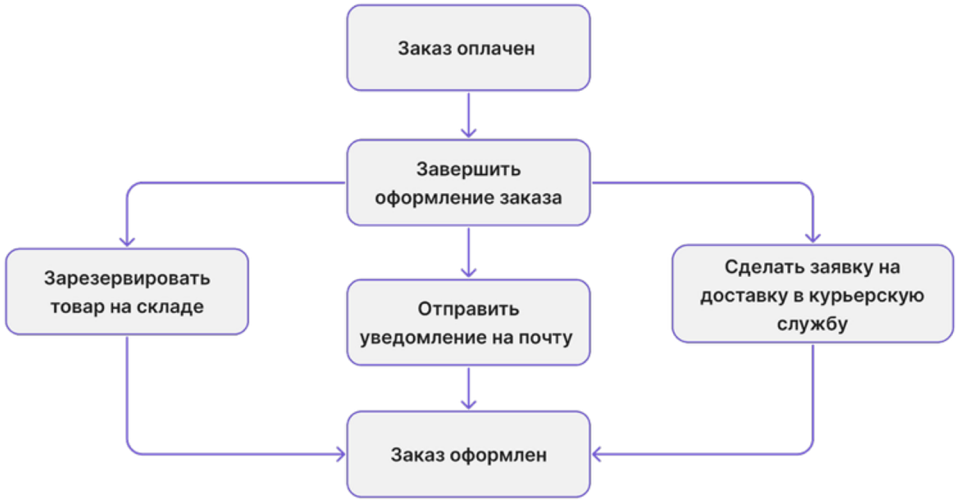
Как связать сервис заказов и сервис доставки?

- Когда оплата завершилась, нам надо еще

  1) послать уведомление на почту

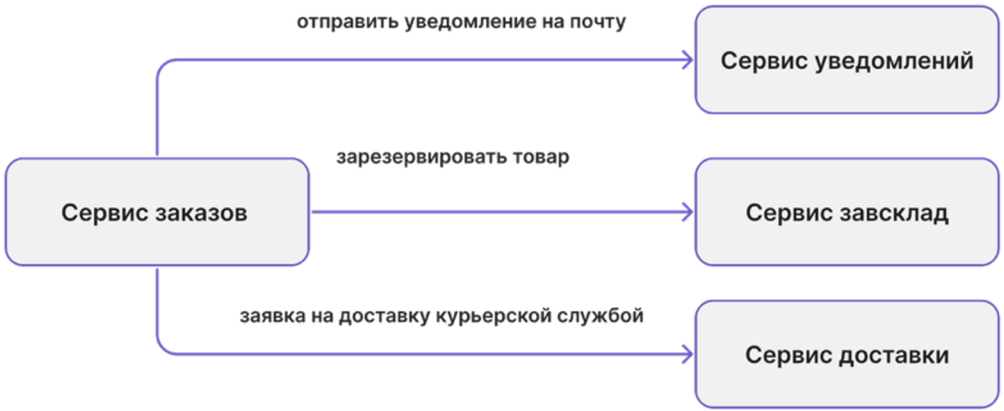
  2) зарезервировать товар на складе /

  3) сделать заявку на доставку в курьерскую службу



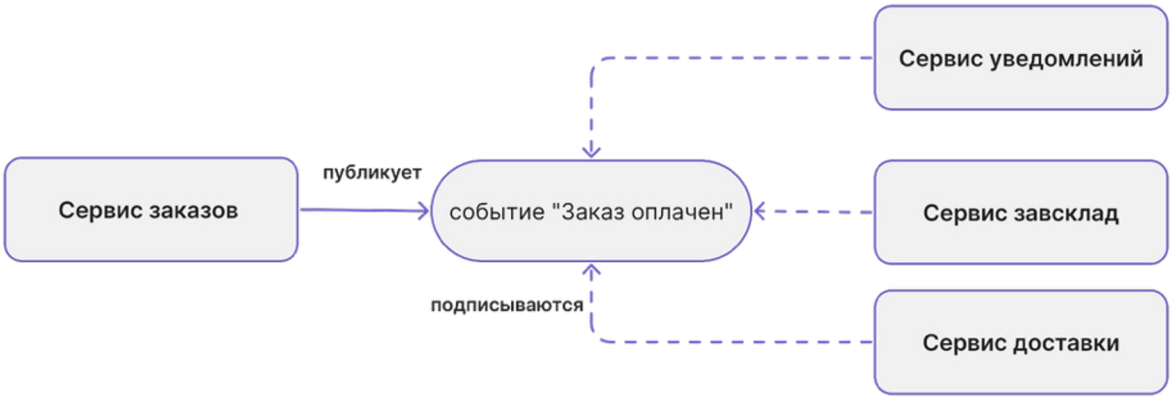
Как связать сервис заказов и сервис доставки?

- Сервис Заказов явным образом (синхронно) ходит во все остальные сервисы, когда получает сообщение из очереди, что заказ оплачен.



Как связать сервис заказов и сервис доставки?

- Сервис Заказов публикует сообщение, что заказ оплачен, а остальные сервисы его слушают.

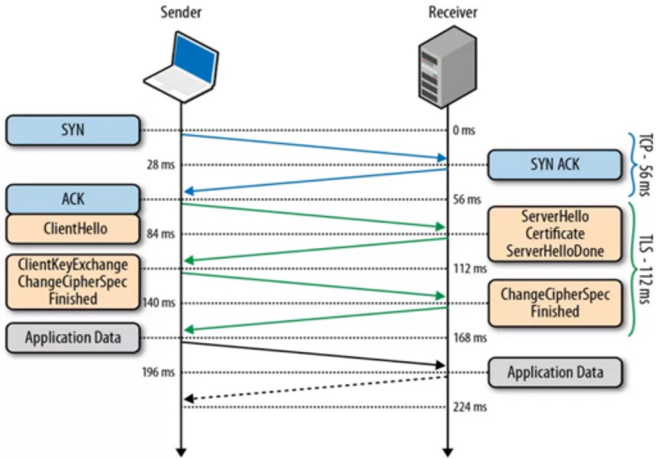


Что еще стоит учесть?

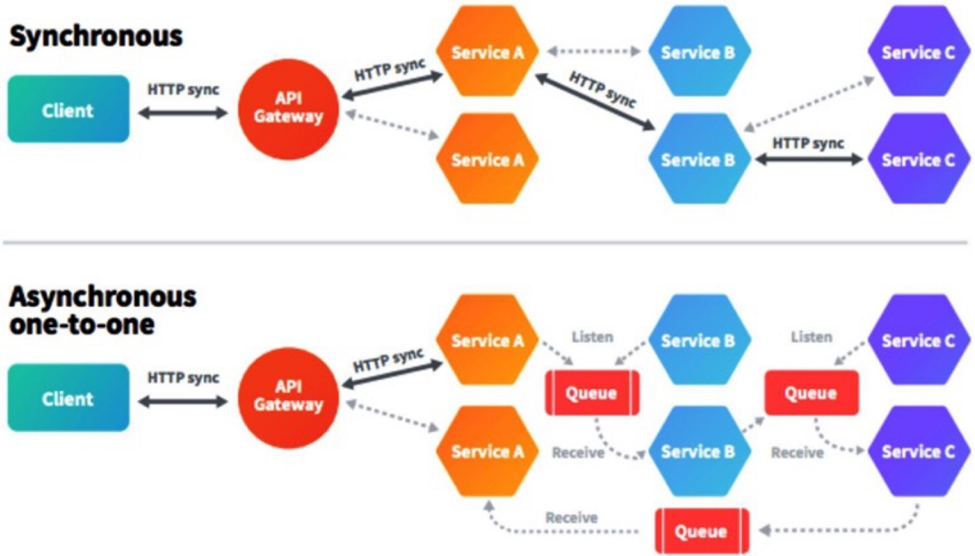
Latency

- Вызов по сети на несколько порядков медленнее, чем вызов локальной функции.

```
Latency Comparison Numbers (~2012)
----------------------------------
L1 cache reference                           0.5 ns
Branch mispredict                            5   ns
L2 cache reference                           7   ns                      14x L1 cache
Mutex lock/unlock                           25   ns
Main memory reference                      100   ns                      20x L2 cache, 200x L1 cache
Compress 1K bytes with Zippy             3,000   ns        3 us
Send 1K bytes over 1 Gbps network       10,000   ns       10 us
Read 4K randomly from SSD*             150,000   ns      150 us          ~1GB/sec SSD
Read 1 MB sequentially from memory     250,000   ns      250 us
Round trip within same datacenter      500,000   ns      500 us
Read 1 MB sequentially from SSD*     1,000,000   ns    1,000 us    1 ms  ~1GB/sec SSD, 4X mem
Disk seek                           10,000,000   ns   10,000 us   10 ms  20x datacenter round
Read 1 MB sequentially from disk    20,000,000   ns   20,000 us   20 ms  80x memory, 20X SSD
Send packet CA->Netherlands->CA    150,000,000   ns  150,000 us  150 ms
```



Синхронное взаимодействие ухудшает доступность



Согласованность и согласованное представление

Оформление заказа

1. Оплата
2. Формирование заказа в сервисе Kitchen
3. Планирование доставки в сервисе Delivery

Повествование - занятие по распределенным транзакциям.

Согласованное представление

Сколько еще надо продуктов на оплаченные, но недоставленные заказы?

Рекомендации

1. Начинайте с основного пользовательского сценария.
2. Исходя из пользовательского сценария с помощью одного из методов постройте модель предметной области.
3. Из модели предметной области сделайте нулевую итерацию разбиения сервисов
4. Пройдитесь по всем сценариям и для каждого сервиса попробуйте конкретно описать его взаимодействие с клиентом, и с другими сервисами. Например, используя MicroserviceCanvas.
5. Оцените сервисную модель: какие у нее есть слабые стороны, проблемы, какие возникли вопросы в рамках составления карточки сервисов? Как с точки зрения функциональных, так и не функциональных требований.
6. Придумайте решение проблем текущего разбиения, переразбейте сервисы, по-другому организуйте взаимодействие и т.д.
7. Повторяйте п. 4 - 6 до полного просветления.

Подведём итоги занятия

Слайд с тезисами

Подведем итоги

1. Хорошо понимать предметную область(бизнес-домен), которую проектируем.
2. Монолит сначала, переход к микросервисной архитектуре в дальнейшем.
3. Разбиение на сервисы (один домен на сервис). Разбиение по агрегатам, функциональное моделирование. Разбиение бизнес-возможностям, транзакциям. Event storming.
4. API и описание микросервиса: Microservices Canvas, bounded context canvas.
5. Что стоить учесть: возможные изменения, latency, доступность и другие нефункциональные требования.

ДЗ «Паттерны декомпозиции микросервисов»

Домашнее задание
«Паттерны декомпозиции микросервисов»

В качестве архитектурной задачи возьмем кейс интернет-магазина. Ката ***I'll Have the BLT*** по ссылке https://nealford.com/katas/list.html
Во всех последующих ДЗ будем работать над данным кейсом.

Для архитектурной задачи используйте один из паттернов декомпозиции и предложите вариант разбиения на микросервисы.

В качестве решения предоставьте:
Пользовательские сценарии.
Схему взаимодействия сервисов – C4 model контейнерная диаграмма (см. ссылки на доп.источники по C4 model ниже). Описание каждого микросервиса по одному из шаблонов на выбор (в материалах занятия), включая:
Для каждого микросервиса опишите его назначение и зону ответственности.
Опишите контракты взаимодействия микросервисов друг с другом.

Заполните, пожалуйста, опрос о занятии

Мы читаем все ваши сообщения
и берем их в работу ✏️❤️